Part 1 exercises: predict, change, observe, explain

Keep your editor and terminal next to each other. Restart the agent after editing Python files so your changes take effect.

1. Observe one tool call

Run `python try_tools.py`. Find these four things in `tools.py` and the output:

1. The tool name: `save_recommendation`.
2. Its input: `recommendation`, which is text.
3. Its action: save that text to a fixed file.
4. Its returned result: a message the agent can read.

Change the example recommendation in `try_tools.py` and run it again. Predict which output will change before running. The file contents should change; the filename and confirmation message should stay the same.

This test directly calls the tool. No model chooses the action, and the temporary file is removed afterward. It demonstrates the action, not the agent's judgment.

Now run `python agent.py` and ask:

> List the files in data/. Read the customer interviews and suggest one product question. Don't search the web or save a report yet.

Record a tool name, its input, and what information came back. For a file read, the terminal shows its path; open that file to inspect the contents returned to the agent. Explain why that information might help answer your request.

2. Change the agent's behavior and capabilities

Find `Agent(...)` in `create_agent(model)` in `agent.py`. Identify its name, model, instructions, and tools. Then find `Runner.run_sync(...)` in `run_agent`. Explain why constructing an agent and running it are separate steps.

Edit `INSTRUCTIONS` in `agent.py`. Add one specific behavior, for example:

> Before recommending an improvement, explain one disagreement between the customer interviews.

Restart and repeat the same request. Record whether the answer follows the new instruction and what evidence it uses. One successful run is an observation, not a guarantee that it will always comply.

Next remove `search_web` from the `TOOLS` list. Restart and ask it

> Check a current competitor feature on the web.

It should explain the limitation; check whether it instead guesses. Restore `search_web` afterward.

3. Write a local search tool

In `tools.py`, read the working `search_local(query, path)` reference, then write your own matching body. Keep a copy of the reference to compare afterward. Keep its decorator and signature; it is already registered in `agent.py`.

Use `documents = files.read_texts(path)` to obtain a dictionary of file names (keys) and their contents (values) such as:

```
{"data/example.md": "Import needs support.\nCRM later."}
```

The library handles folder traversal and file limits. Your code decides which files match and what the agent gets back. Implement these steps:

1. Convert the entire query to lowercase with `query.lower()`. Keep it as one string: `"IMPORT preview"` becomes `"import preview"`.
2. Loop through the filenames and text in alphabetical filename order.
3. Use `query in content.lower()` to check whether the entire query appears in each file's text. Keep the filename when it does.

4. Return a list of up to five filenames, each appearing only once, such as `["data/example.md"]`.
5. Return `[]` for an empty or whitespace-only query, or no matches. Use `if query.strip():` before the loop to skip searching for a blank query.

This searches for one continuous piece of text. `"import preview"` matches `"Import preview"`, but not `"import a preview"`, `"preview import"`, or `"import\npreview"`. Spaces, line breaks, and punctuation must match exactly. Partial words still count: `"import"` also matches `"imported"` and `"important"`.

The docstring describes this behavior to the agent. Put explanations for students in ordinary # comments; these are not part of the tool description.

Keep `text_arg` for input limits and `files.show_result` for terminal output. Those are supplied helpers; the matching and returned list are your code. LLM help is allowed, but explain each line and predict another input's result.

In `try_tools.py`, import `search_local` and try:

```
try_tool(search_local, query="import", path="data")
try_tool(search_local, query="IMPORT", path="data")
try_tool(search_local, query="import preview", path="data")
try_tool(search_local, query="", path="data")
try_tool(search_local, query="zzznomatchzzz", path="data")
```

Compare case handling, entire-query matching, and empty results. Open one returned file and find the entire query in its contents, ignoring case.

Ask the agent to

> We're considering improving CSV import before the Singapore pilot. Which customer actually needs this, and what evidence suggests it should—or shouldn't—be our priority? Find relevant passages in the local materials and any saved research notes. Cite filenames and interview IDs, and distinguish customer requests from existing capabilities.

Explain which folder the model chose.

4. Inspect a plan

First read `update_plan` in `tools.py`, then follow its `save_plan` call into `plans.py`. A plan has `goal`, `steps`, and `revision_note`; each `PlanStep` has `id`, `description`, and `status`. Find the validation rules, Markdown conversion, and save operation. A future interface can call the same functions. Each call replaces the entire plan, so include unchanged steps too.

Try this without a model (import `update_plan` in `try_tools.py`):

```
try_tool(update_plan, goal="Understand onboarding", steps=[
    {"id": "s1", "description": "Read interviews", "status": "in_progress"},
    {"id": "s2", "description": "Compare needs", "status": "pending"},
], revision_note="Initial plan")
```

Predict the saved Markdown. The temporary tester starts fresh on each call; use the live agent below to observe successive revisions of one saved plan.

Edit `INSTRUCTIONS` in `agent.py` to tell the agent to make, update, and follow a plan. Add the following instruction (if it is already present, keep one copy):

```
Use update_plan for multi-step work when helpful or requested.
When a plan exists, use its active step to guide your next actions. Mark a step
done only after checking its result, then move to the next step. If the task or
evidence changes, revise the plan before continuing.
```

Restart the agent so the instruction takes effect.

Choose a product question. Ask the agent:

> Help me understand major customer pain points. Read the relevant files in data/. Make a short visible plan before investigating my question. Save reusable notes and a short report. Use only the supplied files for now.

Keep the question in the same conversation. Inspect `runtime/plan.md` and the saved report. Give a changed requirement, for example:

Focus on small businesses with limited technical support. Search your saved notes, revise the plan, and recommend just one improvement.

Compare the plan and report before and after. Identify a meaningful change in scope or action, not just a status changing to `done`. Check that completed steps have corresponding evidence or saved output.

Ask one follow-up of your own: change a criterion, correct an assumption, or ask why it made a recommendation. Optional: permit a small public web search and compare the external evidence with the fictional classroom materials.

Extend this code for your project

For project ideas and code entry points, see EXTENDING.md.